

Contents

03-Credentials and App Lifecycle Guide	1
Credentials storage and Slack app lifecycle implementation guide	1
1. Purpose and implementation boundary	1
2. The objects that Slack and the framework manage	2
3. Credential types and what each authorizes	3
4. Trace the implemented manifest and creation path	4
5. Proposed persistent identity and schema	7
6. File storage, locking, and recovery	9
7. Refresh is a state machine with a remote commit	10
8. Proposed CLI and app-creation lifecycle	12
9. Installation and runtime provisioning	13
10. Proposed Go package boundaries and API shape	15
11. Implementation phases and acceptance criteria	16
12. Verification and operational diagnostics	17
13. Decisions, alternatives, and deferred questions	18
14. API and source reading index	19

03-Credentials and App Lifecycle Guide

Credentials storage and Slack app lifecycle implementation guide

1. Purpose and implementation boundary

This document specifies a local credential manager for the Go Slack framework and explains how it connects to manifest generation, app creation, installation, and runtime authentication. The intended reader is an intern who understands basic Go functions, JSON, and HTTP but has not implemented Slack authentication. Read the identity model before implementing file storage: choosing the wrong record key can cause a correctly implemented HTTP client to authenticate against the wrong workspace.

The recommended first implementation stores non-secret selection metadata in `~/.config/go-go-slack/config.yaml` and secrets in private files below `credentials/`. Profiles select existing records; they do not copy tokens. Management credentials are keyed independently from apps, and bot credentials are keyed by an installation's app and workspace. Each rotating access/refresh pair occupies one record that is replaced atomically under a process-shared lock.

This is a design proposal with a small implemented foundation. At the time of writing, bots manifest NAME generates a manifest, and the working tree implements bots create-app NAME using a configuration access-token file. Profile storage, refresh, OAuth callback handling, and production Slack runtime authentication are **not implemented**. The local Socket Mode runner accepts only a loopback mock. Creating an app does not make that runner a production client.

The guide's examples use fictional IDs such as `A_EXAMPLE` and `T_DEV`. They illustrate relations and are not valid credentials. Proposed commands are explicitly labeled. Nothing in this document requires printing, archiving, or committing an actual secret.

1.1 Learning and review sequence

1. Understand app registration, installation, and token ownership in sections 2–3.
2. Trace the implemented command from JavaScript to HTTP in section 4.
3. Implement record validation and local persistence before refresh in sections 5–7.
4. Connect lifecycle commands and installation using sections 8–10.
5. Use the implementation phases and failure tests in sections 11–12 as acceptance criteria.

The earlier architecture guide explains the JavaScript host and ingress in depth. The local testing plan explains SDK/mock evaluation. This document concentrates on credential ownership and app provisioning.

2. The objects that Slack and the framework manage

A **workspace** is a Slack collaboration environment identified by a team ID. Its display name is useful to a person but is not a durable database key. A user can belong to several workspaces, and two workspaces can have similar names. Store Slack IDs as strings without trying to derive their meaning from names or token prefixes.

An **app registration** is a Slack application definition with an app ID, configuration, and app-level credentials. Its manifest describes requested features and permissions. A registration can subsequently have installations in multiple workspaces, subject to Slack’s distribution and administrative policies. The development workspace associated with registration does not identify every installation.

An **installation** is an authorization of an app in a workspace. It carries granted permissions and the resulting bot credentials. Updating the desired scopes in a manifest does not by itself establish that an existing installation has granted those scopes. Desired configuration and granted authorization are distinct state.

A **local bot definition** is the JavaScript source loaded by this repository. Its name, such as `ping`, selects local code. It is not a Slack app ID. The same source can support separate development and production registrations; conversely, one registration can have multiple installations running the same definition.

A **profile**, in this proposal, is a human-friendly selection of records: which management identity, app registration, installation, and local bot to use. A profile alias is convenient CLI input. It is not an authentication principal and must not be embedded into record keys that need to

survive renaming.

Local bot: ping

```
|
+--> Profile: ping-dev
|     management --> user U_OWNER in T_DEV
|     app -----> A_DEV
|     installation -> (A_DEV, T_DEV)
|
+--> Profile: ping-production
|     management --> user U_OWNER in T_DEV
|     app -----> A_PROD
|     installation -> (A_PROD, T_PROD)
```

App A_PROD

```
+--> app credentials and Socket Mode token
+--> installation (A_PROD, T_PROD): bot token
+--> installation (A_PROD, T_OTHER): another bot token
```

This separation prevents three common implementation errors: using a bot token to create an app, using one installation's token for another workspace, and duplicating an app-level Socket Mode token into every installation record. It also makes a missing prerequisite explainable: a profile can know an app ID before it has an installation.

2.1 Supported scope of the first manager

The first release should support workspace installations on Slack's standard commercial endpoints, one local operating-system user, and a local filesystem with reliable file locks and rename semantics. It should reject unsupported enterprise-wide installation responses explicitly. Do not collapse an absent team ID into a default workspace. Supporting organization installations later requires a deliberate installation-key design and authorization-routing review.

The manager is separate from the official Slack CLI. It must not parse undocumented Slack CLI login files or depend on a successful `slack login`. Both tools can coexist, but direct API authentication is explicit and owned by this framework.

3. Credential types and what each authorizes

Credentials differ by issuer, owner, purpose, and lifetime. Store the credential kind as a required schema field. Token prefixes can help diagnose an obvious import mistake, but they are not sufficient proof of identity, permissions, or expiration.

Configuration access and refresh pair

- Owner/key: Workspace plus managing user
- Purpose: Manifest management APIs
- Acquisition and renewal: App Configuration Tokens dashboard; `tooling.tokens.rotate`

App client secret

- Owner/key: App ID
- Purpose: Authenticate OAuth exchanges
- Acquisition and renewal: App credentials returned at creation or app settings

Signing secret

- Owner/key: App ID
- Purpose: Verify signed inbound HTTP requests
- Acquisition and renewal: App credentials or settings

Socket Mode app token

- Owner/key: App ID plus local token label
- Purpose: Open Socket Mode connections
- Acquisition and renewal: App settings, with `connections:write`

Bot access token and optional refresh token

- Owner/key: App ID plus installation workspace
- Purpose: Bot Web API operations
- Acquisition and renewal: OAuth installation; optional OAuth rotation

The app configuration access token is obtained at Your Apps, under **Your App Configuration Tokens → Generate Token**, selecting a workspace. Slack documents these tokens as belonging to a user/workspace pair and expiring after twelve hours. The accompanying refresh token is used with `tooling.tokens.rotate`; the response contains a replacement access token, replacement refresh token, identity fields, and issue/expiry timestamps. The import UI should accept both files and preserve the pair. See the archived configuration guide and rotation method.

`app_configurations:write` is the permission used for app creation. Adding it to the bot's requested OAuth scopes is not a way to issue a management credential. A bot access token authorizes runtime work for an installation, while the configuration token authorizes management work. Keep management secrets out of a bot runtime process when only message delivery is needed.

The Socket Mode app token is also distinct from a bot token. Opening a connection and making a bot Web API call require different credentials. A signing secret verifies inbound HTTP signatures; it is not a bearer token for either operation. Preserve a returned legacy verification token only when importing the original response for recovery; do not implement new request verification with it.

Bot-token rotation is a separate protocol from configuration-token rotation. When enabled, OAuth access credentials expire and are refreshed through `oauth.v2.access`, using the application's OAuth credentials. The manager must record whether rotation is enabled and use returned expiration metadata. Do not assign a twelve-hour expiry to an imported non-rotating bot token. Enabling rotation is an explicit app policy change, not a storage initialization side effect. See Slack's rotation guide and its local snapshot.

4. Trace the implemented manifest and creation path

All repository paths in this section are relative to the `discord-bot` root. Symbols are included so references remain useful when line numbers change. These paths describe real source files, not the proposed package layout later in the guide.

`cmd/slack-bot/main.go`

- Responsibility: Root command, help, logging
- Why read it: Binary entry point

`pkg/slackcli/commands.go`: `NewBotsCommand`, `RunIntoWriter`, `Manifest`

- Responsibility: Glazed fields, command dispatch, manifest generation
- Why read it: Public flags and local-to-Slack translation

pkg/slackcli/create_app.go: configAccessToken, createApp, appHTTPClient

- Responsibility: Explicit token read and direct create request
- Why read it: Existing network and secret boundary

pkg/slackcli/create_app_test.go

- Responsibility: Injected HTTP transport and command tests
- Why read it: Exact request shape, redaction, overwrite prevention

pkg/slackbot/model.go

- Responsibility: Descriptor and host-facing service types
- Why read it: Credential-free domain model

pkg/slackhost/host.go: Inspect, Load

- Responsibility: Public JavaScript host entry points
- Why read it: Trusted source inspection and execution

internal/slacktransport/client.go: NewLocal

- Responsibility: Loopback-only SDK client
- Why read it: Existing transport restrictions

internal/slacktransport/run.go

- Responsibility: Socket Mode lifecycle
- Why read it: Later production credential injection boundary

pkg/slackdoc/slack-offline.md

- Responsibility: Embedded CLI documentation
- Why read it: Update alongside public command behavior

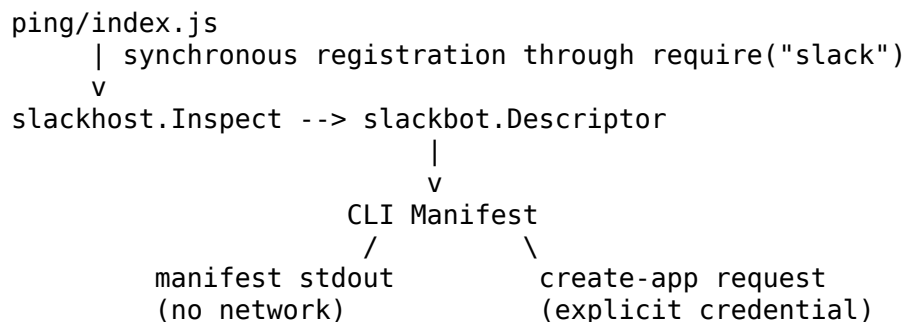
examples/slack-bots/ping/index.js

- Responsibility: Representative bot
- Why read it: Concrete input for all examples

4.1 From JavaScript to a manifest

The CLI resolves the bot name within a local repository and inspects the JavaScript definition. Registration produces a descriptor of commands, events, and declared configuration. Manifest translates that descriptor into Slack configuration, including slash commands, subscribed bot events, bot scopes, Socket Mode, and interactivity settings. Inspection executes trusted

local code; it is not static parsing and is not an untrusted-code sandbox.



Both verbs must continue using the same generator. A separate hand-maintained manifest for creation would allow the reviewed output and submitted app to diverge. Future work should add deterministic manifest hashing and record the submitted hash with the resulting app. The hash documents which configuration was submitted; it is not a Slack idempotency key.

```
go run ./cmd/slack-bot bots manifest ping
```

```
go run ./cmd/slack-bot bots create-app ping \
--config-token-file /tmp/access-token.txt \
--credentials-file /tmp/ping-app-credentials.json \
--timeout-ms 30000
```

These are current command forms. Use the ticket's scripts/04-go-offline.sh run - buildvcs=false in place of go run when the workspace needs the pinned cached toolchain. The word "offline" in that wrapper describes dependency resolution; it does not disable network operations performed by the program.

4.2 The HTTP contract

The creation endpoint is POST <https://slack.com/api/apps.manifest.create>. The existing client supplies Authorization: Bearer <configuration-access-token> and JSON content type. Its JSON body contains an API argument named manifest whose value is the serialized manifest. Sending the unwrapped generated manifest as the entire API body is a different request and should not be used.

```
manifestBytes = JSON.encode(Manifest(descriptor))
body = JSON.encode({manifest: string(manifestBytes)})
response = POST(createEndpoint, bearerToken, body)
require response.HTTPStatus == 200
require response.JSON.ok == true
require response.JSON.app_id is present
```

Slack's documented successful result includes the app ID, app credentials, and an OAuth authorization URL. It does not complete installation or provide the runtime's bot and Socket Mode tokens. HTTP 200 alone is not success because Slack reports application errors in ok and error. See the create API and archived response contract.

The implemented command prints only app ID, settings URL, OAuth authorization URL, and an optional credential-file path. With --credentials-file, it reserves a new file using exclusive creation and mode 0600 before contacting Slack. This avoids creating a remote app when the destination already exists. A failed request can leave that reservation empty. The current writer does not implement the durable record replacement protocol proposed below, and the command has no profile import or token refresh.

The client disables redirects and environment proxy discovery, bounds the response to one MiB, and applies a request deadline. It does not automatically retry creation. A lost response might mean Slack created the app even though the caller received an error. The error therefore directs the operator to inspect app settings before trying again. Do not replace this with a generic retry middleware when adding profiles.

5. Proposed persistent identity and schema

The manager should have one explicitly resolved root directory. On this Linux development workstation, the normal root is `/home/manuel/.config/go-go-slack`. Expose `--config-dir` as a Glazed field for alternate roots and isolated tests. A later cross-platform default may use the operating system's configuration-directory convention, but any environment-based resolution must be documented before implementation. Do not implicitly search environment variables for tokens.

<code>~/.config/go-go-slack/</code>	<code>0700</code>
<code>config.yaml</code>	<code>0600</code>
<code>credentials/</code>	<code>0700</code>
<code>management/<record-id>.json</code>	<code>0600</code>
<code>apps/<record-id>.json</code>	<code>0600</code>
<code>installations/<record-id>.json</code>	<code>0600</code>
<code>locks/</code>	<code>0700</code>
<code>management-<record-id>.lock</code>	<code>0600</code>
<code>app-<record-id>.lock</code>	<code>0600</code>
<code>installation-<record-id>.lock</code>	<code>0600</code>
<code>config.lock</code>	<code>0600</code>
<code>operations/</code>	<code>0700</code>
<code><operation-id>.json</code>	<code>0600</code>

Generate opaque local record IDs, validate their syntax, and derive paths internally. Do not concatenate a user-entered profile name into a filesystem path. Keep lock files stable across record replacement: locking the credential file itself is insufficient because an atomic rename changes the inode another process would lock.

5.1 Metadata selects records

The following is a proposed schema, not a file currently read by the CLI. Metadata can contain IDs and scopes but no access tokens, refresh tokens, client secrets, signing secrets, or response

URLs. It is still private because it reveals workspace and account relationships.

```
version: 1
default_profile: ping-dev
profiles:
  ping-dev:
    bot: ping
    management: management-01
    app: app-01
    installation: installation-01
management:
  management-01:
    team_id: T_DEV
    user_id: U_OWNER
    identity_status: verified
apps:
  app-01:
    app_id: A_EXAMPLE
    development_team_id: T_DEV
    manifest_sha256: EXAMPLE_DIGEST
installations:
  installation-01:
    app: app-01
    team_id: T_DEV
    is_enterprise_install: false
    granted_bot_scopes: [chat:write, commands]
```

The manager enforces uniqueness on (team_id, user_id) for management identities and (app_id, team_id) for supported installations. Several profiles may refer to one record. Deleting a profile removes a selection, not a remote app or shared credential. Explicit record deletion checks references first. Revocation and uninstallation are separate remote operations and must not happen as an incidental consequence of local cleanup.

Do not treat user-entered identity labels as verified facts. Imports initially carry identity_status: unverified unless a trusted API response supplies the binding. Configuration-token rotation returns team and user IDs. An installation OAuth response supplies app and team identity; an additional auth.test check can verify runtime workspace/user information. Each API proves only the fields it actually returns. A token-prefix check proves none of them.

5.2 One management credential record

```
{
  "version": 1,
  "kind": "management",
  "generation": 4,
  "team_id": "T_DEV",
  "user_id": "U_OWNER",
  "access_token": "REDACTED",
  "refresh_token": "REDACTED",
  "issued_at": "2026-09-14T12:00:00Z",
  "expires_at": "2026-09-15T00:00:00Z",
  "status": "ready"
}
```

The timestamps above are illustrative. Convert actual iat and exp from Slack's response to UTC instants. If an import supplies no expiry, store an explicit unknown value rather than

guessing from file modification time. Before managed creation with unknown expiry, the proposed manager should rotate once under lock when a refresh token is available. Otherwise it should require a new import or an explicit file-based request; it must not silently present the token as fresh.

generation advances with every successful credential replacement and is useful for detecting stale snapshots. It is not a substitute for locking. Never write the new access token and refresh token in separate updates. If a process observes a new access token with an old refresh token, its next renewal may fail even though both writes individually succeeded.

5.3 App and installation records

An app record stores its app ID, client ID, client secret, signing secret, and explicitly named Socket Mode app tokens. The client ID is an identifier rather than a secret, but keeping related OAuth fields together simplifies a consistent credential load. Store multiple named app tokens only when there is a real need to select among them; a profile can reference a label without duplicating values.

An installation record stores app ID, team ID, bot user ID, granted scopes, bot access token, and optional refresh/expiry fields. A `rotation_mode` field must distinguish none, oauth, and unknown. Do not infer a valid refresh workflow merely because a field happens to contain a string. Reject a profile whose installation references a different app from the selected app record.

Secret records repeat identity fields from metadata intentionally. On load, compare them. A mismatch is a hard error that names the record IDs and conflicting non-secret IDs. Never “repair” a mismatch by overwriting one side automatically; that can hide a mistaken import.

6. File storage, locking, and recovery

The first backend should be private files, with an interface narrow enough to test failure boundaries. An operating-system keychain can be a future backend if requested, but an encrypted JSON file with its encryption key stored next to it provides little additional protection against the same filesystem reader. Avoid adding a keychain abstraction before the record and lifecycle contracts are stable.

The file backend protects against accidental repository inclusion and ordinary access by other OS users. It does not isolate secrets from root, malware running as the same user, or a debugger attached to the process. This limitation defines what the permission checks accomplish; it is not a reason to expose secrets in logs or environment variables.

6.1 Reading and importing

For a managed root, verify that directories and files are owned by the expected user and reject group/world-accessible secret files. On the initial Linux implementation, reject symlinks at managed path components and use descriptor-based no-follow opening where necessary to avoid check/open races. Check file type, cap record size, decode exactly one JSON value, reject unknown schema versions, and validate required identity fields before returning a record.

Import commands should accept explicit file paths, not literal token flags that enter shell history and process arguments. The two paths already supplied by the user are suitable bootstrap inputs. Importing copies their contents into a validated private record; it should not delete the input files without a separate requested cleanup. A management import must read both inputs before attempting any mutation, and it must never include either value in diagnostic errors.

6.2 Durable replacement protocol

Atomic rename prevents readers from seeing a partly written JSON document. Synchronization adds durability against crashes. These are separate properties. The proposed Linux file backend follows this order:

```
replaceRecord(recordID, expectedGeneration, next):
    acquire stable lock for recordID, respecting context
    current = readAndValidate(recordID)
    require current.generation == expectedGeneration
    validate next identity and next generation
    temp = createExclusiveRandomFileInSameDirectory(0600)
    writeAll(temp, JSON(next))
    sync(temp)
    closeAndCheck(temp)
    rename(temp, finalPath)
    sync(parentDirectory)
    release lock
```

For refresh, the caller already holds the record lock throughout the remote operation; the storage helper therefore needs an explicit locked transaction API rather than recursively acquiring the lock above. Ordinary readers can open a complete snapshot without a lock, but any decision to refresh must acquire the lock and reread first. Keep the network timeout bounded so another CLI process is not blocked indefinitely.

If write or sync fails before rename, the old record remains authoritative. Remove only the temporary file owned by that operation. If directory sync fails after rename, report that the visible update succeeded but durable persistence is uncertain; do not restore an older refresh token. Include safe record identity and recovery instructions, never serialized record content.

Metadata and credential files cannot be atomically replaced together using a single rename. Store a newly created credential record first, then update metadata under `config.lock`. A crash can leave an orphan credential file, which is recoverable by identity. Writing metadata first can create a reference to credentials that do not exist. A `credentials doctor` command should report orphans without deleting them, and a recovery command should require matching IDs before linking them.

6.3 Locks and process lifetime

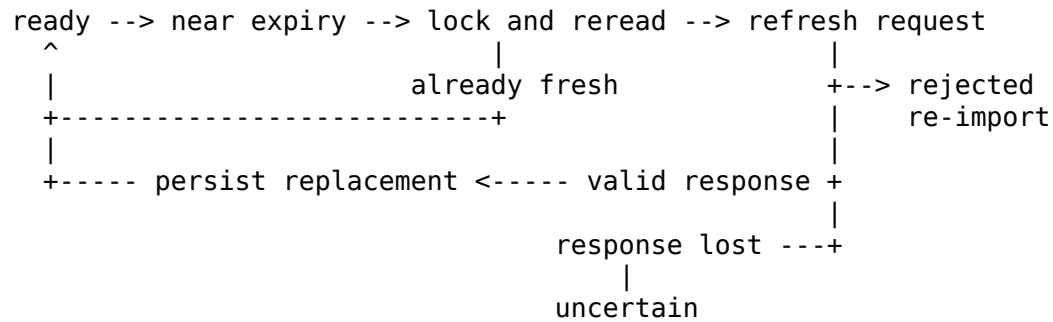
Use advisory OS locks on stable lock files, not PID-file existence as a mutex. The OS releases a process's lock on exit. A leftover lock filename therefore does not indicate a live owner. Tests must exercise two independent processes; a Go mutex alone cannot coordinate two terminals running the CLI.

Avoid holding `config.lock` during HTTP calls. Resolve the target record first, release metadata access, acquire the specific credential lock, and then refresh. If an operation needs several record locks, define and enforce one lexical ordering by record ID. The first implementation should keep most operations to one credential lock to reduce deadlock risk. Network filesystems and cross-host refresh coordination are outside the first backend's support contract.

7. Refresh is a state machine with a remote commit

Configuration refresh changes state at Slack and state on disk. No local filesystem transaction can make those two changes atomic. This is the most important failure boundary in the

manager: receiving new tokens and persisting them are separate events.



The proposed freshness policy uses a small clock-skew allowance, for example two minutes, and the time needed for the intended operation. This is a local policy, not a Slack guarantee. Inject a clock into the manager so tests can control expiration without waiting. A caller asks for enough validity for its request deadline rather than merely checking whether expiry is later than the current instant.

```
configurationAccess(ctx, recordID, minimumValidity):
    lock(recordID, ctx)
    record = readValidated(recordID)
    if record.expiry > now + minimumValidity + skew:
        return accessToken(record)
    require refresh token exists
    persist safe operation state "refresh-in-flight"
    result = toolingRotateOnce(ctx, record.refreshToken)
    if response is ambiguous:
        mark recovery required; return safe error
    require result.ok and both replacement tokens
    require result.teamID == record.teamID
    require result.userID == record.userID
    require result.expiry > now + skew
    replaceLocked(recordID, result, generation + 1)
    mark operation complete
    return result.accessToken
```

An interrupted refresh must not be blindly repeated with the old refresh token. The response might have been lost after Slack changed its state. The first implementation should surface a recovery-required state and direct the operator to generate/import a new configuration pair when the replacement cannot be recovered. Do not claim the operation journal can reconstruct a secret response that was never persisted.

The journal records record ID, generation, operation ID, timestamps, and status only. On restart, an unfinished refresh journal entry blocks automatic renewal until reconciliation. If the credential generation is already newer and identity matches, the replacement completed locally and the journal can be marked complete. If it did not advance, the outcome remains uncertain. A crash before the HTTP request actually left the process can therefore conservatively require recovery; this trades convenience for avoiding unsupported refresh replay assumptions.

OAuth bot refresh uses a separate client method and record kind. It must not call `tooling.tokens.rotate`. Its implementation needs the app's client credentials plus the installation refresh token, and it must preserve the installation binding when updating the result. Reuse the storage transaction mechanism, but keep protocol validation specific to each credential type.

8. Proposed CLI and app-creation lifecycle

The initial public path remains `bots create-app NAME`, because that command already means “generate this bot’s manifest and create its Slack registration.” Add a profile input to that verb rather than introducing another command with the same behavior. Proposed commands below illustrate the intended interface; they do not exist yet.

```
slack-bot credentials import-management \
  --profile ping-dev \
  --access-token-file /tmp/access-token.txt \
  --refresh-token-file /tmp/refresh-token.txt

slack-bot profiles list
slack-bot profiles show ping-dev
slack-bot bots create-app ping --profile ping-dev
slack-bot credentials status --profile ping-dev
slack-bot credentials refresh --profile ping-dev
slack-bot apps link --profile ping-dev --app-id A_EXAMPLE
```

Selection precedence should be explicit profile, then configured default profile. If neither exists, fail with an actionable message. Do not silently choose the only file in a directory. Reject combining `--profile` and `--config-token-file`; both designate authentication sources. A profile-based create should persist returned credentials automatically to the private store, while the existing explicit-file command keeps its documented output-file behavior. This is a deliberate new mode, not an adapter to the Slack CLI’s storage.

Before creating, ensure the profile does not already reference an app. Creating a second registration should require another profile or an explicit new-app operation. A valid local bot name and a valid token do not establish that creating another app is intended.

8.1 Prepare, validate, create, persist

The proposed managed creation flow is:

1. Resolve the profile and verify its management identity.
2. Inspect the bot once, generate the manifest, and compute its canonical hash.
3. Check local destination capacity and permissions; reserve an operation ID.
4. Obtain a sufficiently fresh management access token under its record lock.
5. Call `apps.manifest.validate` with the exact manifest to be submitted.
6. Record a safe create-in-flight operation and call `create` once.
7. Persist the app credential record, then link metadata and the profile.
8. Emit app ID, settings URL, operation ID, manifest hash, and installation status.

Validation is proposed here; the current `create` command directly calls `create`. Server-side validation can explain schema errors before mutation, but it does not reserve a name, guarantee a later `create` will succeed, or make the operation idempotent. The validation method snapshot

is the contract to use when adding it.



Creation errors need distinct statuses. `rejected` means Slack returned a definitive error. `outcome_unknown` means the response did not establish whether an app exists. `created_unlinked` means the app ID is known but local persistence or profile linking failed. The last two must never be disguised as ordinary retryable failures.

Recovery begins with the known app ID or Slack’s app dashboard. Link an existing app after verifying the intended registration. The operation ID is a local correlation value, not a server-supported request deduplication key. An app name alone is insufficient evidence that a discovered registration belongs to a particular lost request.

8.2 Updating a manifest is a different action

Do not make `create-app` silently update an existing app. A future update verb should require an app ID, show or record the prior and desired manifest, call the relevant update API, and explain when changed permissions require another installation authorization. Neither an update nor an export implementation is included in the current code or this phase’s minimum acceptance criteria.

9. Installation and runtime provisioning

After creation, the app exists but does not yet have a bot authorization for a selected workspace. The current command returns Slack’s OAuth URL for the user to follow. A future managed installation flow can open an authorization URL and receive a registered callback, but the user’s consent remains a browser interaction; direct APIs do not eliminate that authorization step.

The OAuth flow has three participants: the user’s browser, the Slack authorization service, and a Go callback handler. The handler exchanges a short-lived authorization code for the installation result using `oauth.v2.access`. Associate each attempt with a cryptographically random, expiring, single-use state value and the intended local profile. Reject callbacks with missing, expired, mismatched, or previously consumed state before exchanging a code. Configure an exact allowed redirect URI; do not assume Slack accepts an arbitrary loopback callback. See

installation documentation and the OAuth access method.

```
startInstall(profile):
    require selected app and OAuth credentials
    pending = randomStateBoundTo(profile, expiry, redirectURI)
    persist pending state privately
    open authorization URL for this app and state

callback(code, state):
    pending = consumeValidState(state)
    result = exchangeCodeOnce(code, pending.redirectURI)
    require result.ok
    require result.appID == selected appID
    require supported workspace installation
    require expected workspace, or explicit user selection
    persist installation credentials and granted scopes
    link profile to installation
```

The callback must not log query strings, authorization codes, or raw response bodies. A workspace selected in a browser can differ from the one the CLI operator intended, so check returned identity rather than trusting an authorization URL hint. Initially support bot credentials only; reject or deliberately ignore additional user-token grants without accidentally persisting them as bot credentials. If OAuth policy later adopts PKCE, implement it from Slack's specific documented flow as a separate reviewed change.

Socket Mode still needs an app-level token with `connections:write`. The first manager should import that token from an explicit file and associate it with the selected app. Do not promise a public API that issues every required credential automatically. App creation, browser installation, and importing a Socket Mode token are separate readiness checks.

9.1 Keep credentials behind Go services

A production runner should resolve the profile once, validate the installation/app relationship, and inject authenticated Go services into the JavaScript host. JavaScript receives methods such as message posting, not tokens or the entire credential record. A rotating runtime client needs an installation-token provider that obtains a fresh token before a request; constructing one static SDK client with an expiring token is insufficient for a long-running process.

```
profile -> app record -----> Socket Mode connection owner
      \-> installation ---> token provider -> Web API client
                                   |
                                   | MessageService interface
                                   |
                                   | JavaScript host
```

Inbound workspace identity must select the matching installation, and admission policy must reject unexpected workspaces before dispatch. For the first production runner, explicitly select one installation and reject others. Later multi-workspace serving requires a routing layer keyed by verified workspace/app identity. This prevents a message received for one installation from being sent with another installation's credentials.

The current `NewLocal` transport enforces loopback-only access and should retain that contract. Introduce a separate production construction path when authorized and tested. Do not loosen the mock client's restrictions just because credentials now exist on disk.

10. Proposed Go package boundaries and API shape

Use the existing top-level module. Proposed packages below are implementation locations, not existing files. Follow repository conventions: Glazed fields for CLI inputs, github.com/pkg/errors for contextual errors, context-aware blocking work, zerolog for safe diagnostic fields, and compile-time interface assertions. Do not add environment-token fallback or compatibility layers.

internal/slackcredentials/model.go

- Responsibility: Typed identities, records, profiles, validation

internal/slackcredentials/filestore.go

- Responsibility: Private reads, locks, atomic durable replacement

internal/slackcredentials/manager.go

- Responsibility: Profile resolution, freshness, refresh state machine

internal/slackapp/client.go

- Responsibility: Typed manifest and tooling API calls

internal/slackapp/lifecycle.go

- Responsibility: Validate/create/persist and recovery states

pkg/slackcli/credentials.go

- Responsibility: Glazed import/status/refresh commands

pkg/slackcli/profiles.go

- Responsibility: Profile selection and metadata operations

internal/slackoauth/

- Responsibility: Later authorization attempts and callbacks

Keep packages internal until their contracts need external consumers. The existing creation helper can move into the app client as managed behavior is added; do not maintain two competing HTTP encoders. Keep the manifest generator shared by offline output and app manage-

ment.

// Proposed interfaces; details belong to the implementation phase.

```
type Clock interface {  
    Now() time.Time  
}  
  
type ManagementTokens interface {  
    Access(ctx context.Context, id RecordID,  
        minimumValidity time.Duration) (Secret, error)  
}  
  
type ManifestClient interface {  
    Validate(ctx context.Context, token Secret,  
        manifest []byte) error  
    Create(ctx context.Context, token Secret,  
        manifest []byte) (CreatedApp, error)  
}
```

Secret should have a redacted string representation and no automatic JSON serialization. Its raw value is accessible only to the transport and persistence code that needs it. This reduces accidental logging but cannot prevent all copies in Go memory; do not claim guaranteed zeroization. Return a separate public result type containing safe IDs and status. Never pass a full credential-bearing API response to a generic Glazed row encoder.

The file store should expose an explicit locked transaction or callback so the manager can hold a lock across refresh without copying filesystem knowledge into HTTP code. Implement compile-time assertions such as `var _ ManagementTokens = (*Manager)(nil)`. Use injected HTTP clients, clocks, and narrowly scoped filesystem hooks for deterministic testing, avoiding a large generic plugin framework.

11. Implementation phases and acceptance criteria

Phase A: Finish the direct creation foundation

Review `create_app.go` and its command tests, document that one successful invocation creates one app, and preserve the safe output boundary. Verify manifest encoding against the documented API contract. This phase requires no credential-store schema changes. Its local tests use a synthetic token and injected RoundTripper; they do not establish that a real workspace accepted a manifest.

Phase B: Identity and storage

Implement schema validation, explicit configuration-root selection, profile resolution, private file creation, import, and status. Add generation numbers and consistent app/installation checks before adding remote refresh. Acceptance requires successful isolated-root imports, no token output, rejection of unsupported versions and mismatched identity, and recovery reporting for orphan records. Profile listing must never contact Slack.

Phase C: Configuration refresh

Add the typed tooling client and state machine with a fake clock and process-shared locks. Verify that two processes requesting renewal produce one successful rotation and both subsequently observe the new record. Include interrupted-request and interrupted-persistence

cases. Do not proceed to automatic create until uncertain refresh outcomes are represented explicitly.

Phase D: Managed app creation

Add --profile, mutual exclusion with file auth, remote validation, manifest hashing, operation records, app persistence, and profile linking. Reject profiles already linked to apps. Acceptance requires one create request on success, no create after failed local preparation, and a created_unlinked recovery path that does not repeat the request.

Phase E: Installation and production readiness

Implement explicit bot-token and Socket Mode-token import first if that meets local development needs. Add OAuth callbacks as a separate phase with state and identity tests. Only then add a production runner and rotating Web API authentication. Keep multi-workspace routing out of the first runner; make the selected installation an explicit invariant.

Each phase should be an independently reviewable commit with a diary entry describing its behavior, evidence, limitations, and next step. Do not combine a new storage format, OAuth server, token-refresh daemon, and production transport into one unreviewable change.

12. Verification and operational diagnostics

Most tests need no real credentials. Temporary directories, fake clocks, and injected HTTP transports cover the credential manager's state transitions. Use subprocess tests for locks and crash recovery, because those properties are not demonstrated by concurrent goroutines alone.

Same app installed in two workspaces

- Required observation: Distinct installation records and token selection

Two profiles reference one manager

- Required observation: One shared refresh generation, no duplicated pair

Profile renamed

- Required observation: Credential paths and Slack identities unchanged

Wrong app/workspace import

- Required observation: Rejected before profile linking or runtime HTTP

Empty, oversized, malformed, symlink input

- Required observation: Safe error; no credential content in output

Existing create output file

- Required observation: No HTTP request

Expiry within skew window

- Required observation: One refresh before dependent request

Two CLI processes refresh

- Required observation: One replacement; waiter rereads fresh state

Refresh succeeds, local write fails

- Required observation: Recovery required; no blind use of old pair

Create response is lost

- Required observation: Outcome unknown; no automatic second creation

App record saved, metadata write fails

- Required observation: Orphan discoverable and linkable by identity

OAuth callback replay

- Required observation: Rejected before another code exchange

API returns HTTP 200 with ok:false

- Required observation: Typed failure, no success record

Debug logging enabled

- Required observation: No access/refresh tokens, codes, or app secrets

For the current foundation, run the focused `pkg/slackcli` and `cmd/slack-bot` tests and compilation. For future storage implementation, test durability failure points using injected failures before write completion, after file sync, after rename, and before directory sync. A unit test cannot prove behavior under every filesystem or physical power failure; document the supported platform and use process termination tests to exercise observable recovery.

Real Slack acceptance should be separate and explicit: import a test management identity, validate a manifest, create exactly one app, record its safe app ID, install it in the chosen test workspace, and import the app-level Socket Mode token. Sending a test message is a subsequent runtime test with a chosen channel. Configuration tokens are enough for creation but not for that runtime test.

Diagnostics should expose profile name, record ID, Slack IDs, credential kind, generation, expiry, granted scopes, and recovery status. They should omit token suffixes as well as full values: operators can identify records by IDs without leaking credential fragments. `credentials` status must not refresh or mutate silently. `credentials` refresh is explicit; normal API operations may refresh only through the documented manager policy.

13. Decisions, alternatives, and deferred questions

The initial choice is private files with normalized records because it matches local CLI development and makes backup, inspection of safe metadata, and isolated tests straightforward. One `.env` file per bot fails to represent shared management identities and app-level credentials cleanly, and it encourages process-wide implicit selection. A single large secret JSON file would simplify whole-store replacement but serialize unrelated refreshes and expose every credential to readers that need only one. Per-record storage reduces that coupling while requiring explicit metadata recovery.

A keychain backend may become appropriate for interactive desktop use, and a centralized secret service may be appropriate for multi-host deployments. Neither is required to implement the local protocol correctly. The API should not pretend that a local lock coordinates refresh across hosts; a distributed backend needs its own conditional-write or lease contract.

Deferred decisions are enterprise installation support, GovSlack endpoints, multi-host refresh ownership, unattended OAuth callback hosting, and a public external credential-provider interface. They should be resolved when there is a concrete deployment need. The first implementation can proceed without them by enforcing the narrower scope above.

14. API and source reading index

Public pages were extracted with defuddle into the ticket's `sources/app-creation/` directory. Its README links all archived sources; `manifest.json` records URLs, retrieval timestamps, byte counts, and SHA-256 hashes. Earlier Socket Mode, manifest schema, event, and scope references remain in the parent `sources` directory. The archives contain public documentation only.

Configuration identity and bootstrap

- Public reference: Manifest configuration guide
- Local snapshot: `01-configuration-tokens.md`

Create registration

- Public reference: `apps.manifest.create`
- Local snapshot: `02-create-api.md`

Validate manifest

- Public reference: `apps.manifest.validate`
- Local snapshot: `03-validate-api.md`

Renew management pair

- Public reference: `tooling.tokens.rotate`
- Local snapshot: `04-rotate-token-api.md`

Manual app/token setup

- Public reference: App setup
- Local snapshot: `05-app-setup.md`

Official CLI API invocation

- Public reference: slack api
- Local snapshot: `07-cli-api.md`

Browser authorization

- Public reference: Installing with OAuth
- Local snapshot: `16-oauth-installation.md`

Code exchange and bot refresh

- Public reference: `oauth.v2.access`

- Local snapshot: 17-oauth-access-api.md

Installation rotation policy

- Public reference: Using token rotation
- Local snapshot: 18-oauth-token-rotation.md

Read the local snapshots when working offline. Before implementing an additional endpoint or relying on an undocumented retry property, verify the current official contract and archive that source too. The proposed schemas and state machines in this guide are framework design decisions; the referenced Slack documents specify the remote API behavior.